

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI *DOUBLE LINKED LIST (DLL)*
MENGGUNAKAN BAHASA *JAVA*



Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 6 dengan judul “Implementasi *Double Linked List (DLL)* Menggunakan Bahasa *Java*” tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep struktur data *Double Linked List (DLL)* serta penerapannya dalam pemrograman *Java*. Melalui praktikum ini, penulis mempelajari bagaimana merepresentasikan data dalam bentuk *node* yang saling terhubung dengan *pointer*, serta memahami cara kerja struktur data *linear* dinamis.

Selain itu penulis juga mengimplementasikan berbagai operasi dasar pada *Double Linked List*, seperti penambahan data (*insert*) di awal, akhir, dan posisi tertentu, penghapusan data (*delete*) pada berbagai kondisi, serta penelusuran data (*traversal*) secara *forward traversal* maupun *backward traversal*. Dengan implementasi ini, penulis dapat memahami alur manipulasi data dalam struktur linked list secara lebih mendalam.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 13 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan	6
1.3 Manfaat	6
BAB II PEMBAHASAN.....	7
2.1 Dasar Teori.....	7
2.1.1 Operasi Dasar	8
2.2 Langkah Kerja	9
2.2.1 Pembuatan <i>Package</i> dan <i>Class</i>	9
2.2.2 Membuat Program <i>NodeDLL_2511532002</i>	13
2.2.3 Membuat Program <i>InsertDLL_2511532002</i>	14
2.2.4 Membuat Program <i>PenelusuranDLL_2511532002</i>	18
2.2.5 Membuat Program <i>HapusDLL_2511532002</i>	20
2.2.6 <i>Upload</i> ke GitHub	25
2.3 Hasil.....	28
2.3.1 Hasil <i>NodeDLL_2511532002</i>	28
2.3.2 Hasil <i>insertDLL_2511532002</i>	28
2.3.3 Hasil <i>PenelusuranDLL_2511532002</i>	29
2.3.4 Hasil <i>HapusDLL_2511532002</i>	29
BAB III KESIMPULANN	30
3.1 Kesimpulan	30
3.2 Saran	30

DAFTAR PUSTAKA.....	31
TUGAS PRAKTIKUM PEKAN 5.....	32
1. Kode Program	32
1.1 Kelas <i>Lagu</i>_2511532002	32
1.2 Kelas Musik_2511532002	33
1.3 Kelas MainProgram_2511532002.....	38
2. Hasil.....	42
3. Kesimpulan	43

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengelolaan data yang efisien dan terstruktur dalam proses komputasi. Salah satu aspek penting dalam pemrograman adalah penggunaan struktur data yang tepat untuk mengatur dan memanipulasi data sesuai dengan kebutuhan sistem. Salah satu struktur data yang banyak digunakan adalah *Linked List*, khususnya *Double Linked List (DLL)*, yang berperan dalam pengelolaan data secara dinamis.

Double Linked List (DLL) merupakan struktur data linear yang terdiri dari sekumpulan node yang saling terhubung, di mana setiap *node* memiliki tiga bagian, yaitu data, pointer yang menunjuk ke node berikutnya (*next*), dan *pointer* yang menunjuk ke *node* sebelumnya (*prev*). Berbeda dengan *Single Linked List (SLL)* yang hanya memiliki satu arah penelusuran, *Double Linked List* memungkinkan proses traversal dilakukan secara maju maupun mundur sehingga lebih fleksibel dalam manipulasi data.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar *Double Linked List*, cara implementasinya dalam bahasa *Java*, serta penerapan berbagai operasi dasar seperti penambahan (*insert*), penghapusan (*delete*), dan penelusuran (*traversal*). Dengan pemahaman tersebut, penulis diharapkan mampu memilih dan menggunakan struktur data yang tepat untuk menyelesaikan permasalahan secara efektif dan efisien.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

- 1.2.1 Memahami konsep dasar struktur data *Double Linked List (DLL)* sebagai salah satu struktur data linear dinamis.
- 1.2.2 Mengetahui struktur *node* pada *Double Linked List* yang terdiri dari data, *pointer* ke *node* berikutnya (*next*), dan *pointer* ke *node* sebelumnya (*prev*).
- 1.2.3 Mengimplementasikan operasi dasar pada *Double Linked List* menggunakan bahasa Java, meliputi penambahan data (*insert*), penghapusan (*delete*), dan penelusuran data (*traversal*).
- 1.2.4 Melatih kemampuan dalam menganalisis alur manipulasi data pada struktur *Double Linked List* untuk menyelesaikan permasalahan pemrograman secara efektif.

1.3 Manfaat

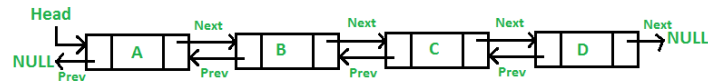
Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

- 1.3.1 Menambah pemahaman mengenai penerapan struktur data *Double Linked List* dalam pemrograman *Java*.
- 1.3.2 Membantu mahasiswa memahami cara pengelolaan data secara dinamis tanpa harus menentukan ukuran data di awal.
- 1.3.3 Melatih kemampuan logika pemrograman dalam melakukan operasi penambahan, penghapusan, dan penelusuran data.

BAB II

PEMBAHASAN

2.1 Dasar Teori



Double Linked List (DLL) merupakan salah satu jenis struktur data linear dinamis yang terdiri dari sekumpulan *node* yang saling terhubung. Setiap *node* pada *Double Linked List* memiliki tiga bagian utama, yaitu data, *pointer* ke *node* berikutnya (*next*), dan *pointer* ke *node* sebelumnya (*prev*). Dengan adanya dua *pointer* tersebut, proses penelusuran data dapat dilakukan ke dua arah, yaitu maju (*forward*) dan mundur (*backward*).

Berbeda dengan *Single Linked List* yang hanya memiliki satu arah penelusuran, *Double Linked List* memberikan fleksibilitas lebih tinggi dalam manipulasi data karena setiap *node* mengetahui *node* sebelum dan sesudahnya. Struktur dasar *Double Linked List* dapat digambarkan sebagai berikut:

$$NULL \leftarrow Node1 \leftrightarrow Node2 \leftrightarrow Node3 \rightarrow NULL$$

Node pertama memiliki nilai *prev* = *null* karena tidak memiliki *node* sebelumnya, sedangkan *node* terakhir memiliki nilai *next* = *null* karena tidak memiliki *node* sesudahnya.

2.1.1 Operasi Dasar

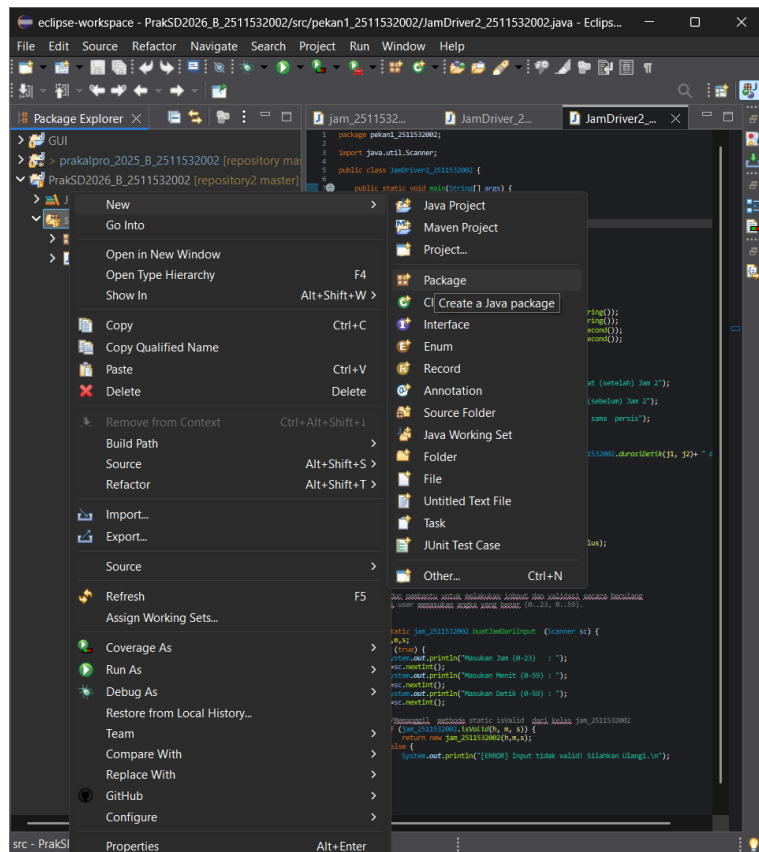
Pada implementasinya, *Double Linked List* memiliki beberapa operasi dasar, yaitu:

1. *Traversal*: *Traversal* merupakan proses penelusuran node secara berurutan, baik dari awal ke akhir maupun dari akhir ke awal.
2. *Insertion*: Operasi penambahan node dapat dilakukan di awal (*insert beginning*), di akhir (*insert end*), maupun pada posisi tertentu dalam *linked list*.
3. *Deletion*: penghapusan node dilakukan dengan mengatur ulang pointer next dan prev agar node yang dihapus tidak lagi terhubung dengan node lainnya.

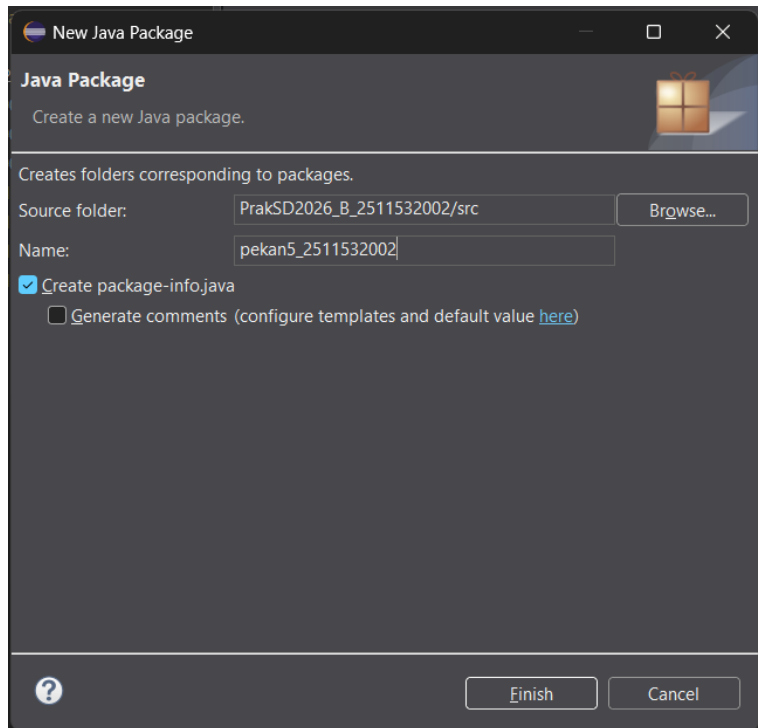
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

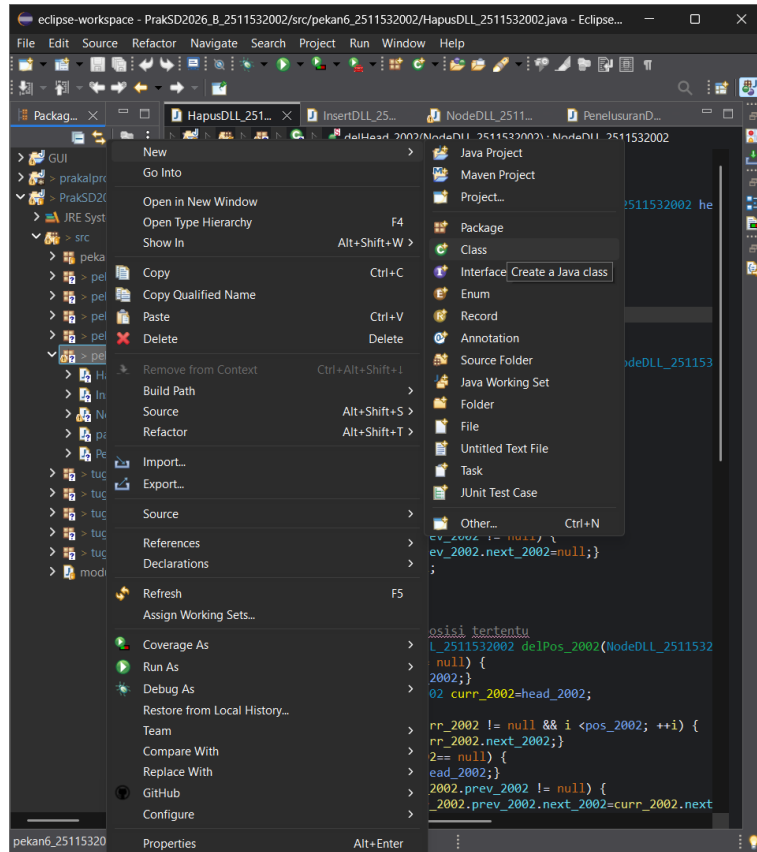
- 1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* “PrakSD2026_B_2511532002” lalu pilih “New” dan klik “Package”.



- 2) Setelah itu akan muncul *pop up* “Java Package”, buat nama *package* dengan ketentuan tanpa *capslock*, *space*, dan karakter khusus lainnya. Lalu klik “*finish*”.



- 3) Selanjutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih “New” dan klik “Class”.



- 4) Setelah itu pada menu “*Java Class*” buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.

New Java Class

Create a new Java class.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass:

Interfaces:

Which method stubs would you like to create?

☐ public static void main(String[] args)

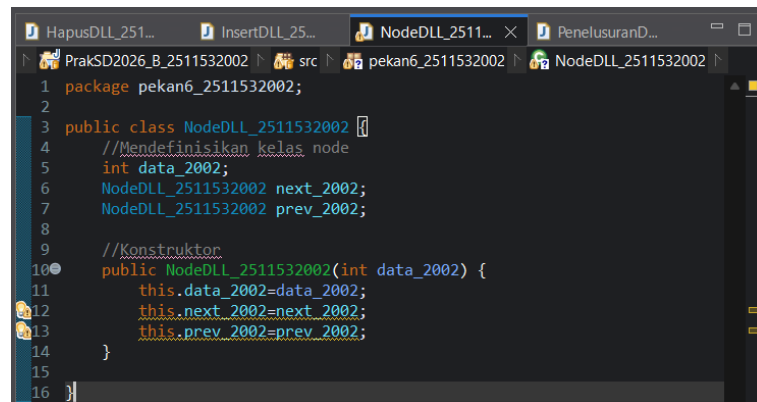
☐ Constructors from superclass

☐ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐

2.2.2 Membuat Program *NodeDLL_2511532002*



```

1 package pekan6_2511532002;
2
3 public class NodeDLL_2511532002 {
4     //Mendefinisikan kelas node
5     int data_2002;
6     NodeDLL_2511532002 next_2002;
7     NodeDLL_2511532002 prev_2002;
8
9     //Konstruktor
10    public NodeDLL_2511532002(int data_2002) {
11        this.data_2002=data_2002;
12        this.next_2002=next_2002;
13        this.prev_2002=prev_2002;
14    }
15
16 }

```

Setelah membuat *class QueueArray_2511532002*, maka mahasiswa dapat menulis *syntax* nya.

1. Menentukan atribut (*Type*) yang digunakan dalam *Node Double*
int data_2002;
NodeDLL_2511532002 next_2002;
NodeDLL_2511532002 prev_2002;
2. Membuat konstruktor untuk menginisialisasi *node*.

```

public NodeDLL_2511532002(int data_2002) {
    this.data_2002=data_2002;
    this.next_2002=next_2002;
    this.prev_2002=prev_2002;
}

```

2.2.3 Membuat Program *InsertDLL_2511532002*

```
package pekan6_2511532002;

public class InsertDLL_2511532002 {
    //Menambahkan node di awal DLL
    static NodeDLL_2511532002 insertBegin_2002(NodeDLL_2511532002
    head_2002, int data_2002) {
        //buat node baru
        NodeDLL_2511532002 new_node_2002 = new
        NodeDLL_2511532002(data_2002);
        new_node_2002.next_2002=head_2002;
        if (head_2002 != null) {
            head_2002.prev_2002=new_node_2002;
        }return new_node_2002;
    }

    //Fungsi menambahkan node di akhir
    public static NodeDLL_2511532002
    insertEnd_2002(NodeDLL_2511532002 head_2002, int
    newData_2002) {
        NodeDLL_2511532002 newNode_2002=new
        NodeDLL_2511532002(newData_2002);
        if (head_2002 == null) {
            head_2002 = newNode_2002;
        }else {
            NodeDLL_2511532002 curr_2002=head_2002;
            while (curr_2002.next_2002 != null) {
                curr_2002=curr_2002.next_2002;
            }
            curr_2002.next_2002=newNode_2002;
            newNode_2002.prev_2002=curr_2002;
        }
        return head_2002;
    }

    // Fungsi menambahkan node diposisi tertentu
    public static NodeDLL_2511532002
    insertAtPosition_2002(NodeDLL_2511532002 head_2002, int
    pos_2002, int new_data_2002) {
        NodeDLL_2511532002 new_node_2002=new
        NodeDLL_2511532002(new_data_2002);
        if (pos_2002 == 1) {
            new_node_2002.next_2002=head_2002;
            if (head_2002 != null) {
                head_2002.prev_2002=new_node_2002;
            }
            head_2002=new_node_2002;
            return head_2002;
        }
        NodeDLL_2511532002 curr_2002=head_2002;

        for (int i = 1; i < pos_2002 -1 && curr_2002 != null;
        ++i) {
            curr_2002=curr_2002.next_2002;}
        if (curr_2002 == null) {
            System.out.println("Posisi tidak ada");
            return head_2002;
        }

        new_node_2002.prev_2002=curr_2002;
        new_node_2002.next_2002=curr_2002.next_2002;
        curr_2002.next_2002=new_node_2002;
        if (new_node_2002.next_2002 != null) {
            new_node_2002.next_2002.prev_2002=new_node_2002;
        }
        return head_2002;
    }

    public static void printList(NodeDLL_2511532002 head_2002) {
        NodeDLL_2511532002 curr_2002 =head_2002;
        while (curr_2002 != null) {
```

```

        System.out.print(curr_2002.data_2002 + " <-> ");
        curr_2002=curr_2002.next_2002;
    }
    System.out.println();
}

//Main Program
public static void main(String[] args) {
    //membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511532002 head_2002= new NodeDLL_2511532002(2);
    head_2002.next_2002 = new NodeDLL_2511532002(3);
    head_2002.next_2002.prev_2002 = head_2002;
    head_2002.next_2002.next_2002= new NodeDLL_2511532002(5);

    head_2002.next_2002.next_2002.prev_2002=head_2002.next_2002;

    //cetak dll awal
    System.out.println("DLL Awal: ");
    printList(head_2002);
    head_2002=insertBegin_2002(head_2002, 1);
    System.out.println(
        "simpul 1 ditambah di awal: ");
    printList(head_2002);

    System.out.println(
        "simpul 6 ditambah di akhir");
    int data_2002=6;
    head_2002=insertEnd_2002(head_2002, data_2002);
    printList(head_2002);

    //menambah node 4 diposisi 4
    System.out.println("Tambah node 4 di posisi 4: ");
    int data2_2002=4;
    int pos_2002=4;
    head_2002= insertAtPosition_2002(head_2002, pos_2002,
    data2_2002);
    printList(head_2002);
}
}

```

1. Membuat method untuk menambahkan node di awal *Double Linked List* (*insertBegin_2002*).

Pertama dibuat *node* baru menggunakan konstruktor *NodeDLL_2511532002*. Kemudian *pointer next_2002* pada *node* baru diarahkan ke *head* lama. Jika *linked list* tidak kosong, maka *pointer prev_2002* pada *head* lama diarahkan kembali ke *node* baru agar hubungan dua arah tetap terhubung. Setelah itu *node* baru dikembalikan sebagai *head* baru dari *linked list*.

```

static NodeDLL_2511532002 insertBegin_2002(NodeDLL_2511532002 head_2002,
int data_2002) {
    //buat node baru
    NodeDLL_2511532002 new_node_2002 = new
    NodeDLL_2511532002(data_2002);
    new_node_2002.next_2002=head_2002;

    if (head_2002 != null) {
        head_2002.prev_2002=new_node_2002;
    }

    return new_node_2002;
}

```

2. Membuat method untuk menambahkan *node* di akhir *Double Linked List* (*insertEnd_2002*) dengan membuat *node* data baru.

```

public                                static                                NodeDLL_2511532002
insertEnd_2002(NodeDLL_2511532002 head_2002, int newData_2002) {
    NodeDLL_2511532002                                newNode_2002=new
NodeDLL_2511532002(newData_2002);

    if (head_2002 == null) {
        head_2002 = newNode_2002;
    }else {
        NodeDLL_2511532002 curr_2002=head_2002;
        while (curr_2002.next_2002 != null) {
            curr_2002=curr_2002.next_2002;
        }
        curr_2002.next_2002=newNode_2002;
        newNode_2002.prev_2002=curr_2002;
    }
    return head_2002;
}

```

3. Membuat *method* untuk menambahkan *node* pada posisi tertentu (*insertAtPosition_2002*). Dengan melakukan *traversal* menuju posisi sebelum *node* yang akan disisipkan menggunakan perulangan *for*. Setelah posisi ditemukan, *pointer prev_2002* dan *next_2002* diatur agar *node* baru tersisip di antara *node* sebelumnya dan *node* sesudahnya.

```

public                                static                                NodeDLL_2511532002
insertAtPosition_2002(NodeDLL_2511532002 head_2002, int pos_2002, int
new_data_2002) {
    NodeDLL_2511532002                                new_node_2002=new
NodeDLL_2511532002(new_data_2002);

    if (pos_2002 == 1) {
        new_node_2002.next_2002=head_2002;

        if (head_2002 != null) {
            head_2002.prev_2002=new_node_2002;
        }

        head_2002=new_node_2002;
        return head_2002;
    }

    NodeDLL_2511532002 curr_2002=head_2002;

    for (int i = 1; i < pos_2002 -1 && curr_2002 != null; ++i) {

```



```

        curr_2002=curr_2002.next_2002;
    }

    if (curr_2002 == null) {
        System.out.println("Posisi tidak ada");
        return head_2002;
    }

    new_node_2002.prev_2002=curr_2002;
    new_node_2002.next_2002=curr_2002.next_2002;
    curr_2002.next_2002=new_node_2002;

    if (new_node_2002.next_2002 != null) {
        new_node_2002.next_2002.prev_2002=new_node_2002;
    }

    return head_2002;
}

```

4. Membuat *method* untuk mencetak isi *Double Linked List* (*printList*).

```

public static void printList(NodeDLL_2511532002 head_2002) {
    NodeDLL_2511532002 curr_2002 =head_2002;

    while (curr_2002 != null) {
        System.out.print(curr_2002.data_2002 + " <-> ");
        curr_2002=curr_2002.next_2002;
    }

    System.out.println();
}

```

5. Membuat program utama (main) untuk menjalankan operasi insert pada *Double Linked List*.

```

public static void main(String[] args) {
    //membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511532002 head_2002= new NodeDLL_2511532002(2);

    head_2002.next_2002 = new NodeDLL_2511532002(3);
    head_2002.next_2002.prev_2002 = head_2002;

    head_2002.next_2002.next_2002= new NodeDLL_2511532002(5);
    head_2002.next_2002.next_2002.prev_2002=head_2002.next_2002;

    //cetak dll awal
    System.out.println("DLL Awal: ");
    printList(head_2002);
}

```

```

head_2002=insertBegin_2002(head_2002, 1);
System.out.println("simpul 1 ditambah di awal: ");
printList(head_2002);

System.out.println("simpul 6 ditambah di akhir");
int data_2002=6;
head_2002=insertEnd_2002(head_2002, data_2002);
printList(head_2002);

//menambah node 4 diposisi 4
System.out.println("Tambah node 4 di posisi 4: ");
int data2_2002=4;
int pos_2002=4;

head_2002=insertAtPosition_2002(head_2002, pos_2002,
data2_2002);
printList(head_2002);
}

```

2.2.4 Membuat Program PenelusuranDLL_2511532002

```

package pekan6_2511532002;

public class PenelusuranDLL_2511532002 {
    static void forwardTravelsal_2002(NodeDLL_2511532002 head_2002) {
        NodeDLL_2511532002 curr_2002=head_2002;
        while (curr_2002 != null) {
            System.out.print(curr_2002.data_2002 + " <-> ");
            curr_2002=curr_2002.next_2002;
        }
        System.out.println();
    }

    //fungsi penelusuran mundur
    static void backwardTravelsal_2002(NodeDLL_2511532002 tail_2002) {
        NodeDLL_2511532002 curr_2002=tail_2002;
        while (curr_2002 != null) {
            System.out.print(curr_2002.data_2002 + " <-> ");
            curr_2002=curr_2002.prev_2002;
        }
        System.out.println();
    }

    public static void main (String[] args) {
        NodeDLL_2511532002 head_2002 =new NodeDLL_2511532002(1);
        NodeDLL_2511532002 second_2002 =new NodeDLL_2511532002(2);
        NodeDLL_2511532002 third_2002 =new NodeDLL_2511532002(3);

        head_2002.next_2002=second_2002;
        second_2002.prev_2002=head_2002;
        second_2002.next_2002=third_2002;
        third_2002.prev_2002=second_2002;

        System.out.println("Penelusuran maju: ");
        forwardTravelsal_2002(head_2002);
        System.out.println("Penelusuran mundur: ");
        backwardTravelsal_2002(third_2002);
    }
}

```

1. Membuat *method* untuk melakukan penelusuran maju pada *Double Linked List* (*forwardTravelsal_2002*). Program menggunakan variabel *curr_2002* yang diawali dari *node head*. Selama *node* masih ada (*curr_2002 != null*), data pada *node* akan ditampilkan ke layar.

```
static void forwardTravelsal_2002(NodeDLL_2511532002 head_2002) {
    NodeDLL_2511532002 curr_2002=head_2002;

    while (curr_2002 != null) {
        System.out.print(curr_2002.data_2002 + " <-> ");
        curr_2002=curr_2002.next_2002;
    }

    System.out.println();
}
```

2. Membuat *method* untuk melakukan penelusuran mundur pada *Double Linked List* (*backwardTravelsal_2002*). Berbeda dengan *traversal* maju, *traversal* mundur dimulai dari *node* terakhir (*tail*). Program menggunakan variabel *curr_2002* yang diisi dengan *node tail*, kemudian data pada setiap *node* ditampilkan ke layar. Setelah itu *pointer* berpindah ke *node* sebelumnya menggunakan *curr_2002.prev_2002* hingga mencapai *node* pertama.

```
static void backwardTravelsal_2002(NodeDLL_2511532002 tail_2002) {
    NodeDLL_2511532002 curr_2002=tail_2002;

    while (curr_2002 != null) {
        System.out.print(curr_2002.data_2002 + " <-> ");
        curr_2002=curr_2002.prev_2002;
    }

    System.out.println();
}
```

3. Membuat *method* utama (*main*) untuk menjalankan proses penelusuran pada *Double Linked List*. Sebagai program utama untuk menguji proses traversal pada *Double Linked List*. Pertama program membuat tiga *node* dengan data 1, 2, dan 3. Selanjutnya setiap *node* dihubungkan menggunakan *pointer next_2002* dan *prev_2002* sehingga membentuk *Double Linked List*.

```
public static void main (String[] args) {
    NodeDLL_2511532002 head_2002 =new NodeDLL_2511532002(1);
    NodeDLL_2511532002 second_2002 =new NodeDLL_2511532002(2);
    NodeDLL_2511532002 third_2002 =new NodeDLL_2511532002(3);

    head_2002.next_2002=second_2002;
    second_2002.prev_2002=head_2002;

    second_2002.next_2002=third_2002;
    third_2002.prev_2002=second_2002;

    System.out.println("Penelusuran maju: ");
    forwardTravelsal_2002(head_2002);

    System.out.println("Penelusuran mundur: ");
    backwardTravelsal_2002(third_2002);
}
```

2.2.5 Membuat Program *HapusDLL_2511532002*

```
package pekan6_2511532002;

public class HapusDLL_2511532002 {
    public static NodeDLL_2511532002 delHead_2002 (NodeDLL_2511532002
head_2002) {
        if (head_2002 == null) {
            return null;
        }
        head_2002=head_2002.next_2002;
        if (head_2002 != null) {
            head_2002.prev_2002=null;
        }
        return head_2002;
    }

    //fungsi menghapus di akhir
    public static NodeDLL_2511532002
dellast_2002(NodeDLL_2511532002 head_2002) {
        if (head_2002 == null) {
            return null;
        }
        if (head_2002.next_2002 == null) {
            return null;
        }
        NodeDLL_2511532002 curr_2002=head_2002;
        while (curr_2002.next_2002 != null) {
            curr_2002 =curr_2002.next_2002;
        }

        //update pointer
        if (curr_2002.prev_2002 != null) {
            curr_2002.prev_2002.next_2002=null;
        }
        return head_2002;
    }
}
```

```

    }

    //fungsi menghapus node posisi tertentu
    public static NodeDLL_2511532002
delPos_2002(NodeDLL_2511532002 head_2002, int pos_2002) {
        if (head_2002 == null) {
            return head_2002;}
        NodeDLL_2511532002 curr_2002=head_2002;

        for (int i=1; curr_2002 != null && i
<pos_2002; ++i) {
            curr_2002=curr_2002.next_2002;}
        if (curr_2002== null) {
            return head_2002;}
        if(curr_2002.prev_2002
!= null) {
            curr_2002.prev_2002.next_2002=curr_2002.next_2002;}
            if (curr_2002.next_2002
!= null) {
                curr_2002.next_2002.prev_2002=curr_2002.prev_2002;}

            if
(head_2002==curr_2002) {
                head_2002=curr_2002.next_2002;}

                return head_2002;
            }

            //fungsi cetak dll
            public static void
printList(NodeDLL_2511532002 head_2002) {
                NodeDLL_2511532002
curr_2002=head_2002;
                while (curr_2002 !=
null) {
                    System.out.print(curr_2002.data_2002+ " <-> ");
                    curr_2002 =
curr_2002.next_2002;
                }
                System.out.println();
            }

            //main program
            public static void main
(String[] args) {
                NodeDLL_2511532002
head_2002=new NodeDLL_2511532002(1);

                head_2002.next_2002=new
NodeDLL_2511532002(2);

                head_2002.next_2002.prev_2002=head_2002;

                head_2002.next_2002.next_2002=new NodeDLL_2511532002(3);

                head_2002.next_2002.next_2002.prev_2002=head_2002.next_2002;

                head_2002.next_2002.next_2002.next_2002=new
NodeDLL_2511532002(4);

                head_2002.next_2002.next_2002.next_2002.prev_2002=head_2002.next_
2002.next_2002;

```

```

        head_2002.next_2002.next_2002.next_2002.next_2002=new
NodeDLL_2511532002(5);

        head_2002.next_2002.next_2002.next_2002.next_2002.prev_2002=
head_2002.next_2002.next_2002.next_2002;

        System.out.println("DLL
Awal: ");

        printList(head_2002);

        System.out.println("Setelah head dihapus: ");

        head_2002=delHead_2002(head_2002);

        printList(head_2002);

        System.out.println("Setelah node terakhir dihapus: ");
        head_2002 =
delLast_2002(head_2002);

        printList(head_2002);

        System.out.println("Menghapus node ke 2: ");

        head_2002=delPos_2002(head_2002, 2);

        printList(head_2002);

    }
}

```

1. Membuat Membuat method untuk menghapus *node* awal (*delHead*) pada *Double Linked List*. Jika *linked list* kosong (*head == null*), maka langsung mengembalikan *null*. Jika tidak, *head* dipindahkan ke *node* berikutnya (*head.next*).

```

public static NodeDLL_2511532002 delHead_2002 (NodeDLL_2511532002
head_2002) {
    if (head_2002 == null) {
        return null;
    }

    head_2002=head_2002.next_2002;

    if (head_2002 != null) {
        head_2002.prev_2002=null;
    }

    return head_2002;
}

```

2. Membuat *method* untuk menghapus *node* terakhir (*delLastNode_2002*).

```

public static NodeDLL_2511532002 delLast_2002(NodeDLL_2511532002
head_2002) {
    if (head_2002 == null) {
        return null;
    }

    if (head_2002.next_2002 == null) {
        return null;
    }
}

```

```

NodeDLL_2511532002 curr_2002=head_2002;

while (curr_2002.next_2002 != null) {
    curr_2002 =curr_2002.next_2002;
}

//update pointer
if (curr_2002.prev_2002 != null) {
    curr_2002.prev_2002.next_2002=null;
}

return head_2002;
}

```

3. Membuat method untuk menghapus node pada posisi tertentu (*delPos_2002*). Melakukan *traversal* menuju posisi *node* yang akan dihapus menggunakan perulangan *for*:

```

public static NodeDLL_2511532002
delPos_2002(NodeDLL_2511532002 head_2002, int pos_2002) {
    if (head_2002 == null) {
        return head_2002;
    }

    NodeDLL_2511532002 curr_2002=head_2002;

    for (int i=1; curr_2002 != null && i <pos_2002; ++i) {
        curr_2002=curr_2002.next_2002;
    }

    if (curr_2002== null) {
        return head_2002;
    }

    if(curr_2002.prev_2002 != null) {
        curr_2002.prev_2002.next_2002=curr_2002.next_2002;
    }

    if (curr_2002.next_2002 != null) {
        curr_2002.next_2002.prev_2002=curr_2002.prev_2002;
    }

    if (head_2002==curr_2002) {
        head_2002=curr_2002.next_2002;
    }

    return head_2002;
}

```

4. Membuat method untuk menampilkan hasil

```
public static void printList(NodeDLL_2511532002 head_2002) {
    NodeDLL_2511532002 curr_2002=head_2002;

    while (curr_2002 != null) {
        System.out.print(curr_2002.data_2002+ " <-> ");
        curr_2002 = curr_2002.next_2002;
    }

    System.out.println();
}
```

5. Membuat main Program untuk pengujian operasi penghapusan. Pertama program membuat *linked list* dengan data 1 <-> 2 <-> 3 <-> 4 <-> 5. Setelah itu isi *linked list* awal ditampilkan menggunakan *method printList*. Kemudian melakukan beberapa operasi penghapusan.

```
public static void main (String[] args) {
    NodeDLL_2511532002 head_2002=new
    NodeDLL_2511532002(1);

    head_2002.next_2002=new NodeDLL_2511532002(2);
    head_2002.next_2002.prev_2002=head_2002;

    head_2002.next_2002.next_2002=new
    NodeDLL_2511532002(3);
    head_2002.next_2002.next_2002.prev_2002=head_2002.next_2
    002;

    head_2002.next_2002.next_2002.next_2002=new
    NodeDLL_2511532002(4);
    head_2002.next_2002.next_2002.next_2002.prev_2002=head_2
    002.next_2002.next_2002;

    head_2002.next_2002.next_2002.next_2002.next_2002=new
    NodeDLL_2511532002(5);
    head_2002.next_2002.next_2002.next_2002.next_2002.prev_200
    2= head_2002.next_2002.next_2002.next_2002;

    System.out.println("DLL Awal: ");
    printList(head_2002);

    System.out.println("Setelah head dihapus: ");
    head_2002=delHead_2002(head_2002);
    printList(head_2002);
}
```



```

System.out.println("Setelah node terakhir dihapus: ");
head_2002 = delLast_2002(head_2002);
printList(head_2002);

```

```

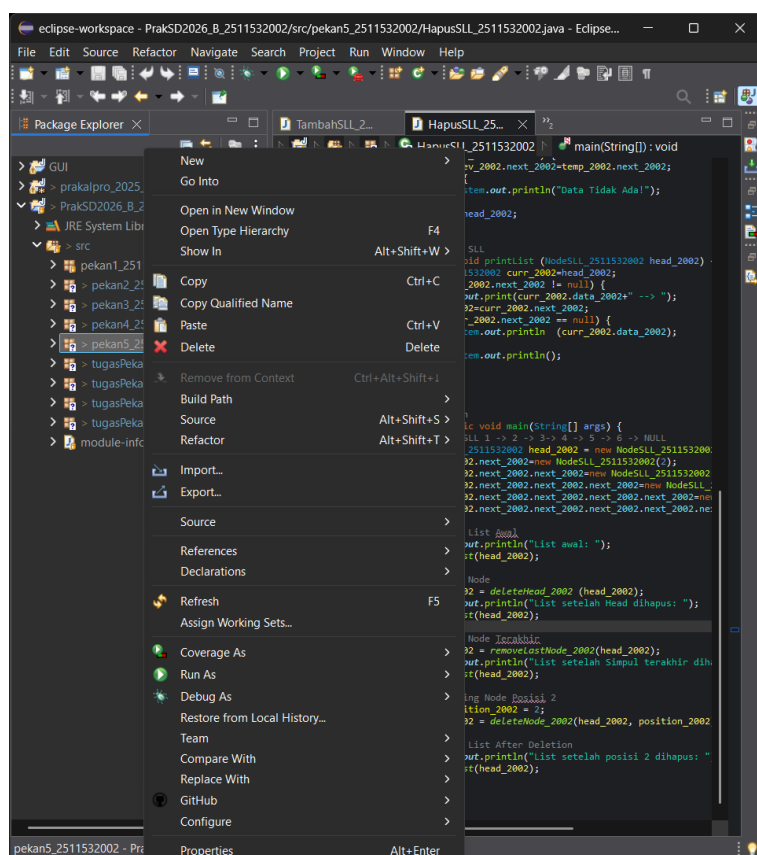
System.out.println("Menghapus node ke 2: ");
head_2002=delPos_2002(head_2002, 2);
printList(head_2002);
}

```

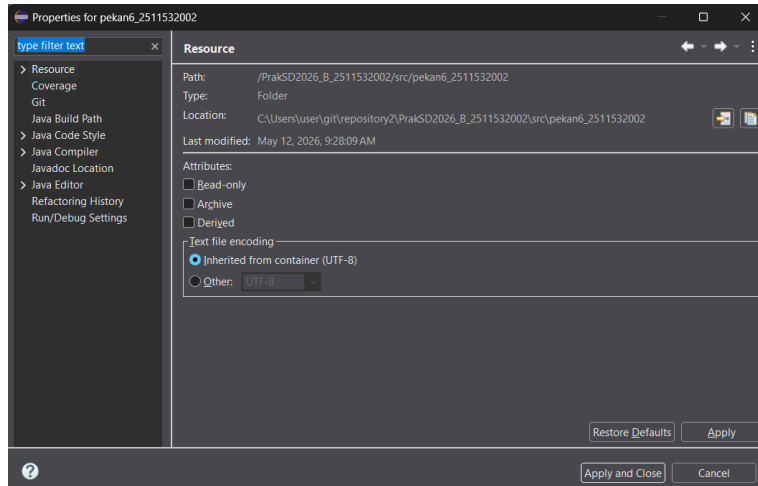
2.2.6 Upload ke GitHub

1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclipse ke *repository* GitHub. Selain dengan cara “*Team*” atau “*Commit and Push*”.

Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package*, lalu pilih *properties*.



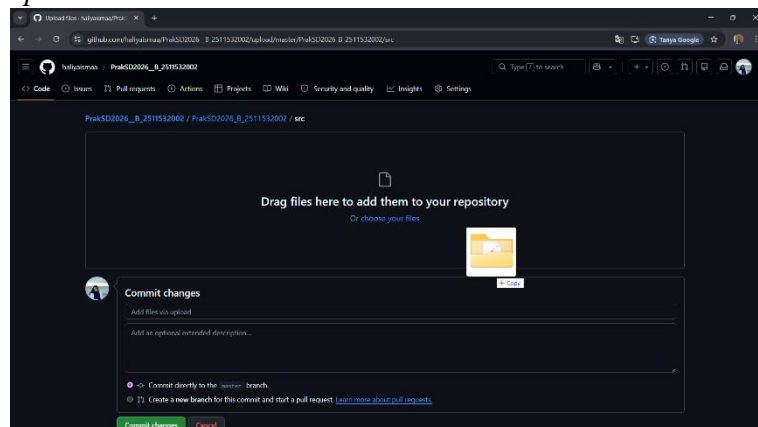
2. Maka akan muncul *pop up menu* “*Properteis for pekan6_2511532002*”. Lalu klik icon “*Show in System Folder*”.



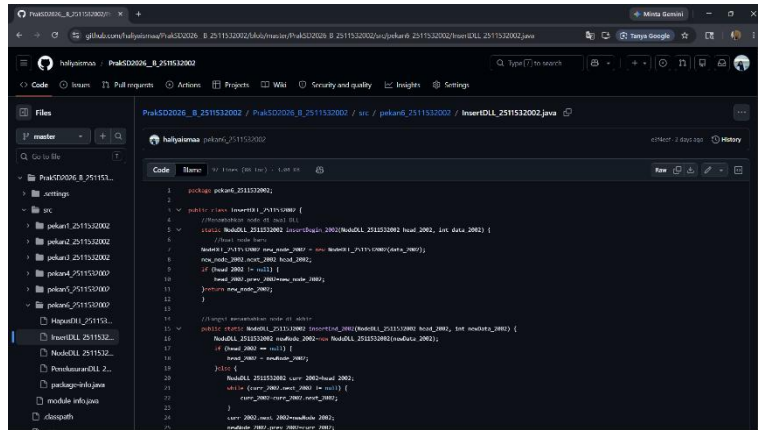
3. Maka akan menunjukan lokasi file “*pekan6_2511532002*” pada *File Explorer computer*.

Name	Date modified	Type
pekan1_2511532002	31/03/2026 09:24	File folder
pekan2_2511532002	06/04/2026 11:42	File folder
pekan3_2511532002	08/04/2026 11:28	File folder
pekan4_2511532002	28/04/2026 09:37	File folder
pekan5_2511532002	05/05/2026 09:28	File folder
pekan6_2511532002	12/05/2026 09:28	File folder

4. Setelah itu *drag and drop* file “*pekan6_2511532002*” ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload*.

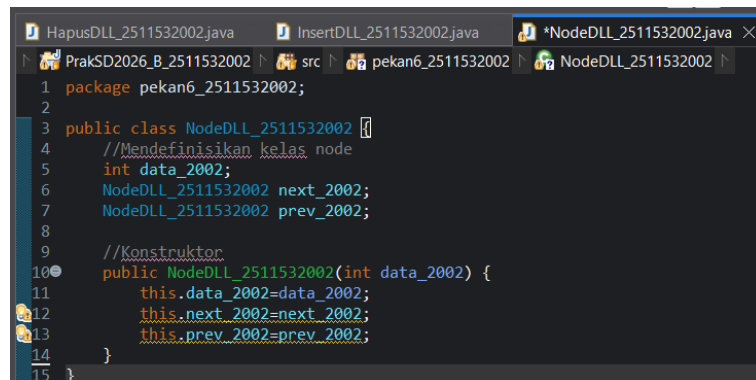


5. Maka dapat dilihat pada *repository GitHub* bahwa file “*pekan6_2511532002*” telah terunggah via *Upload*. Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil *NodeDLL_2511532002*

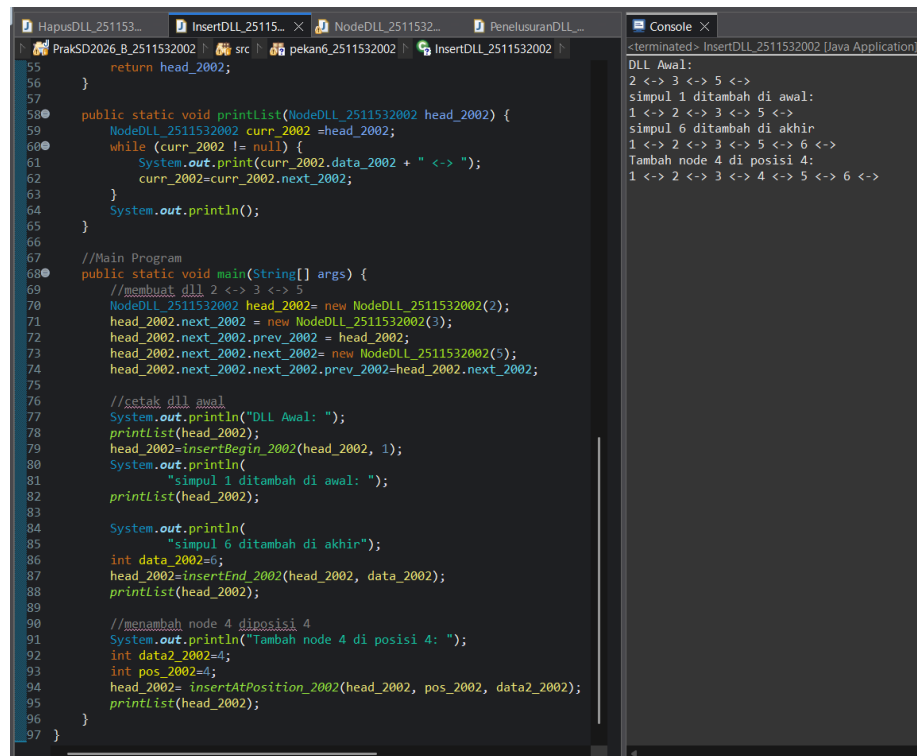


```

1 package pekan6_2511532002;
2
3 public class NodeDLL_2511532002 {
4     //Mendefinisikan kelas node
5     int data_2002;
6     NodeDLL_2511532002 next_2002;
7     NodeDLL_2511532002 prev_2002;
8
9     //Konstruktor
10    public NodeDLL_2511532002(int data_2002) {
11        this.data_2002=data_2002;
12        this.next_2002=next_2002;
13        this.prev_2002=prev_2002;
14    }
15 }

```

2.3.2 Hasil *insertDLL_2511532002*



```

55     return head_2002;
56 }
57
58 public static void printList(NodeDLL_2511532002 head_2002) {
59     NodeDLL_2511532002 curr_2002 = head_2002;
60     while (curr_2002 != null) {
61         System.out.print(curr_2002.data_2002 + " <-> ");
62         curr_2002=curr_2002.next_2002;
63     }
64     System.out.println();
65 }
66
67 //Main Program
68 public static void main(String[] args) {
69     //membuat dll 2 <-> 3 <-> 5
70     NodeDLL_2511532002 head_2002= new NodeDLL_2511532002(2);
71     head_2002.next_2002 = new NodeDLL_2511532002(3);
72     head_2002.next_2002.prev_2002 = head_2002;
73     head_2002.next_2002.next_2002= new NodeDLL_2511532002(5);
74     head_2002.next_2002.next_2002.prev_2002=head_2002.next_2002;
75
76     //setak dll awal
77     System.out.println("DLL Awal: ");
78     printList(head_2002);
79     head_2002=insertBegin_2002(head_2002, 1);
80     System.out.println(
81         "simpul 1 ditambah di awal: ");
82     printList(head_2002);
83
84     System.out.println(
85         "simpul 6 ditambah di akhir");
86     int data_2002=6;
87     head_2002=insertEnd_2002(head_2002, data_2002);
88     printList(head_2002);
89
90     //menambah node 4 diposisi 4
91     System.out.println("Tambah node 4 di posisi 4: ");
92     int data2_2002=4;
93     int pos_2002=4;
94     head_2002= insertAtPosition_2002(head_2002, pos_2002, data2_2002);
95     printList(head_2002);
96 }
97 }

```

Console Output:

```

<terminated> InsertDLL_2511532002 [Java Application]
DLL Awal:
2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal:
1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir:
1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4:
1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

2.3.3 Hasil PenelusuranDLL_2511532002

```

1 package pekan6_2511532002;
2
3 public class PenelusuranDLL_2511532002 {
4     static void forwardTraverseal_2002(NodeDLL_2511532002 head_2002) {
5         NodeDLL_2511532002 curr_2002=head_2002;
6         while (curr_2002 != null) {
7             System.out.print(curr_2002.data_2002 + " <-> ");
8             curr_2002=curr_2002.next_2002;
9         }
10        System.out.println();
11    }
12
13    //fungsi penelusuran mundur
14    static void backwardTraverseal_2002(NodeDLL_2511532002 tail_2002) {
15        NodeDLL_2511532002 curr_2002=tail_2002;
16        while (curr_2002 != null) {
17            System.out.print(curr_2002.data_2002 + " <-> ");
18            curr_2002=curr_2002.prev_2002;
19        }
20        System.out.println();
21    }
22
23    public static void main (String[] args) {
24        NodeDLL_2511532002 head_2002 =new NodeDLL_2511532002(1);
25        NodeDLL_2511532002 second_2002 =new NodeDLL_2511532002(2);
26        NodeDLL_2511532002 third_2002 =new NodeDLL_2511532002(3);
27
28        head_2002.next_2002=second_2002;
29        second_2002.prev_2002=head_2002;
30        second_2002.next_2002=third_2002;
31        third_2002.prev_2002=second_2002;
32
33        System.out.println("Penelusuran maju: ");
34        forwardTraverseal_2002(head_2002);
35        System.out.println("Penelusuran mundur: ");
36        backwardTraverseal_2002(third_2002);
37    }
38 }

```

```

<terminated> PenelusuranDLL_2511532002
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

2.3.4 Hasil HapusDLL_2511532002

```

48 return head_2002;
49
50 }
51
52 //fungsi cetak dll
53 public static void printList(NodeDLL_2511532002 head_2002) {
54     NodeDLL_2511532002 curr_2002=head_2002;
55     while (curr_2002 != null) {
56         System.out.print(curr_2002.data_2002+ " <-> ");
57         curr_2002 = curr_2002.next_2002;
58     }
59     System.out.println();
60 }
61
62 //main program
63 public static void main (String[] args) {
64     NodeDLL_2511532002 head_2002=new NodeDLL_2511532002
65
66     head_2002.next_2002=new NodeDLL_2511532002(2);
67     head_2002.next_2002.prev_2002=head_2002;
68
69     head_2002.next_2002.next_2002=new NodeDLL_2511532002(3);
70     head_2002.next_2002.next_2002.prev_2002=head_2002;
71
72     head_2002.next_2002.next_2002.next_2002=new NodeDLL_2511532002(4);
73     head_2002.next_2002.next_2002.next_2002.prev_2002=
74
75     head_2002.next_2002.next_2002.next_2002.next_2002=new NodeDLL_2511532002(5);
76     head_2002.next_2002.next_2002.next_2002.next_2002.prev_2002=
77
78     System.out.println("DLL Awal: ");
79     printList(head_2002);
80     System.out.println("Setelah head dihapus: ");
81     head_2002=delHead_2002(head_2002);
82     printList(head_2002);
83     System.out.println("Setelah node terakhir dihapus: ");
84     head_2002 = delLast_2002(head_2002);
85     printList(head_2002);
86     System.out.println("Menghapus node ke 2: ");
87     head_2002=delPos_2002(head_2002, 2);
88     printList(head_2002);
89 }
90 }

```

```

<terminated> HapusDLL_2511532002 [Java Application]
DLL Awal:
1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus:
2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus:
2 <-> 3 <-> 4 <->
Menghapus node ke 2:
2 <-> 4 <->

```

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Double Linked List (DLL)* merupakan struktur data linear dinamis yang digunakan untuk menyimpan data menggunakan *node* yang saling terhubung melalui dua *pointer*, yaitu pointer ke node berikutnya (*next*) dan pointer ke node sebelumnya (*prev*). Dengan adanya dua arah penelusuran, pengelolaan data menjadi lebih fleksibel dan efisien.

Dalam praktikum ini, *Double Linked List* berhasil diimplementasikan menggunakan bahasa pemrograman Java dengan berbagai operasi dasar, seperti penambahan data (*insert*), penghapusan data (*delete*), dan penelusuran data (*traversal*) secara maju maupun mundur. Seluruh operasi dapat berjalan dengan baik sesuai konsep *Double Linked List*.

Melalui praktikum ini, penulis dapat memahami cara kerja *Double Linked List* dalam melakukan manipulasi data secara dinamis serta memahami penggunaan pointer antar *node* pada struktur data.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami penulisan *syntax* pemrograman Java. Selain itu, diperlukan ketelitian dalam penulisan *syntax* pemrograman Java, terutama dalam penggunaan *pointer* (referensi) antar *node* dan perulangan agar tidak terjadi kesalahan logika program.

DAFTAR PUSTAKA

[1] *GeeksforGeeks*, “Implementation of Doubly Linked List in JavaScript,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/javascript/implementation-of-doubly-linked-list-in-javascript/>

Diakses pada: 14 Mei 2026.

[2] *Telkom University Jakarta*, “Linked List: Single, Double, dan Circular Linked List,” 2023. [Online]. Available: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>

Diakses pada: 14 Mei 2026.

TUGAS PRAKTIKUM PEKAN 5

1. Kode Program

1.1 Kelas *Lagu_2511532002*

```
package tugasPekan6_2511532002;

public class Lagu_2511532002 {
    String judul_2002;
    String penyanyi_2002;

    Lagu_2511532002 next_2002;
    Lagu_2511532002 prev_2002;

    //Konstruktor
    public Lagu_2511532002(String judul_2002, String
    penyanyi_2002) {
        this.judul_2002=judul_2002;
        this.penyanyi_2002=penyanyi_2002;
        this.next_2002=null;
        this.prev_2002=null;
    }

    //getter
    public String getJudul_2002() {
        return judul_2002;
    }
    public String getPenyanyi_2002() {
        return penyanyi_2002;
    }

    //setter
    public void setJudul_2002(String judul_2002) {
        this.judul_2002=judul_2002;
    }
    public void setPenyanyi_2002(String penyanyi_2002) {
        this.penyanyi_2002=penyanyi_2002;
    }
}
```

Penjelasan :

- 1) *package tugasPekan6_2511532002;*
Menjelaskan posisi file berada pada package bernama “tugaspekan6_2511532002”.
- 2) *public class Lagu_2511532002 {*
Mendefinisikan *class* bernama *Lagu_2511532002* yang digunakan sebagai *node*, pada struktur data *double linked list*.

3) Atribut

- *String judul_2002;*
- *String penyanyi_2002;*

Merupakan atribut dari *class Lagu_2511532002* yang digunakan untuk menyimpan data seperti.

4) Pointer

- *Lagu_2511532002 next_2002;*
- *Lagu_2511532002 prev_2002;*

membuat pointer pada *Double Linked List*. *Pointer next_2002* berfungsi untuk menunjuk ke *node* berikutnya, sedangkan *pointer prev_2002* digunakan untuk menunjuk ke *node* sebelumnya.

5) Konstruktor

```
public Lagu_2511532002(String judul_2002, String  
penyanyi_2002) {  
    this.judul_2002=judul_2002;  
    this.penyanyi_2002=penyanyi_2002;  
    this.next_2002=null;  
    this.prev_2002=null;  
}
```

Merupakan konstruktor yang digunakan untuk mengisialisasikan objek yang digunakan.

1.2 Kelas Musik_2511532002

```
package tugasPekan6_2511532002;  
  
public class Musik_2511532002 {  
    Lagu_2511532002 head_2002;  
    Lagu_2511532002 tail_2002;  
  
    //method tambah lagu di akhir  
    public void tambahLagu_2002(String judul_2002, String penyanyi_2002) {  
        Lagu_2511532002 laguBaru_2002 = new Lagu_2511532002(judul_2002,  
penyanyi_2002);  
  
        //jika playlist kosong  
        if (head_2002 == null) {  
            head_2002 = laguBaru_2002;  
            tail_2002 = laguBaru_2002;  
        } else {  
            tail_2002.next_2002 = laguBaru_2002;  
            laguBaru_2002.prev_2002 = tail_2002;  
            tail_2002 = laguBaru_2002;  
        }  
  
        System.out.println("Lagu berhasil ditambahkan!");  
    }  
  
    //Methode hapu lagu di awal  
    public void hapusLaguAwal_2002() {
```

```

        if (head_2002==null) {
            System.out.println("Playlist Kosong!");
            return;
        }

        System.out.println("Lagu \"" + head_2002.judul_2002 + "\" berhasil
dihapus.");
        head_2002 = head_2002.next_2002;

        if (head_2002 != null) {
            head_2002.prev_2002 = null;
        } else {
            tail_2002 = null;
        }
    }

    //Method tampil maju
    public void tampilMaju_2002() {
        if (head_2002 == null) {
            System.out.println("Playlist kosong!");
            return;
        }

        Lagu_2511532002 curr_2002 = head_2002;
        System.out.println("\n=== Playlist Maju ===");

        while (curr_2002 != null) {
            System.out.println("Judul : " + curr_2002.judul_2002);
            System.out.println("Penyanyi : " +
curr_2002.penyanyi_2002);
            System.out.println();

            curr_2002 = curr_2002.next_2002;
        }

        //Method tampil mundur
        public void tampilMundur_2002() {
            if (tail_2002 == null) {
                System.out.println("Playlist kosong!");
                return;
            }

            Lagu_2511532002 curr_2002 = tail_2002;

            System.out.println("\n=== Playlist Mundur ===");

            while (curr_2002 != null) {
                System.out.println("Judul : " +
curr_2002.judul_2002);
                System.out.println("Penyanyi : " +
curr_2002.penyanyi_2002);
                System.out.println();

                curr_2002 = curr_2002.prev_2002;
            }
        }

        //Method cari lagu
        public void cariLagu_2002(String judul_2002) {
            if (head_2002 == null) {
                System.out.println("Playlist kosong!");
                return;
            }

            Lagu_2511532002 curr_2002 = head_2002;
            boolean ketemu_2002 = false;

            while (curr_2002 != null) {
                if
(curr_2002.judul_2002.equalsIgnoreCase(judul_2002)) {
                    System.out.println("Lagu ditemukan!");
                }
            }
        }
    }

```

```

curr_2002.judul_2002);
curr_2002.penyanyi_2002);
        ketemu_2002 = true;
        break;
    }
    curr_2002 = curr_2002.next_2002;
}
if (!ketemu_2002) {
    System.out.println("Lagu tidak ditemukan!");
}
}
}

```

Penjelasan :

- 1) *package tugasPekan6_2511532002;*
Menjelaskan posisi file berada pada package bernama “tugaspekan5_2511532002”.
- 2) *public class Muik_2511532002 {*
Mendefinisikan *class* bernama *Psien_2511532002* yang digunakan sebagai *node*, pada struktur data *single linked list*.
- 3) *Variabel*
 - *Lagu_2511532002 head_2002;*
 - *Lagu_2511532002 tail_2002;*
Membuat variabel *head_2002* digunakan untuk menyimpan node pertama pada playlist, sedangkan *tail_2002* digunakan untuk menyimpan node terakhir
- 4) Membuat *method* untuk menambahkan lagu di akhir playlist (*tambahLagu_2002*).

```

public void tambahLagu_2002(String judul_2002, String
penyanyi_2002) {
    Lagu_2511532002 laguBaru_2002 = new
    Lagu_2511532002(judul_2002, penyanyi_2002);

    //Jika playlist kosong
    if (head_2002 == null) {
        head_2002 = laguBaru_2002;
        tail_2002 = laguBaru_2002;
    } else {
        tail_2002.next_2002 = laguBaru_2002;
        laguBaru_2002.prev_2002 = tail_2002;
        tail_2002 = laguBaru_2002;
    }
}

```

```
System.out.println("Lagu berhasil ditambahkan!");
}
```

- 5) Membuat *method* untuk menghapus lagu pertama (*hapusLaguAwal_2002*).

```
public void hapusLaguAwal_2002() {
    if (head_2002 == null) {
        System.out.println("Playlist Kosong!");
        return;
    }

    System.out.println("Lagu \"" + head_2002.judul_2002 + "\"
    berhasil dihapus.");
    head_2002 = head_2002.next_2002;

    if (head_2002 != null) {
        head_2002.prev_2002 = null;
    } else {
        tail_2002 = null;
    }
}
```

- 6) Membuat *method* untuk menampilkan playlist secara maju (*tampilMaju_2002*).

```
public void tampilMaju_2002() {
    if (head_2002 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532002 curr_2002 = head_2002;
    System.out.println("\n=== Playlist Maju ===");

    while (curr_2002 != null) {
        System.out.println("Judul : " +
        curr_2002.judul_2002);
        System.out.println("Penyanyi : " +
        curr_2002.penyanyi_2002);
        System.out.println();

        curr_2002 = curr_2002.next_2002;
    }
}
```

- 7) Membuat *method* untuk menampilkan playlist secara mundur (*tampilMundur_2002*).

```

public void tampilMundur_2002() {
    if (tail_2002 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532002 curr_2002 = tail_2002;

    System.out.println("\n=== Playlist Mundur ===");

    while (curr_2002 != null) {
        System.out.println("Judul : " +
            curr_2002.judul_2002);
        System.out.println("Penyanyi : " +
            curr_2002.penyanyi_2002);
        System.out.println();

        curr_2002 = curr_2002.prev_2002;
    }
}

```

8) Membuat *method* untuk mencari lagu (*cariLagu_2002*).

```

public void cariLagu_2002(String judul_2002) {
    if (head_2002 == null) {
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511532002 curr_2002 = head_2002;
    boolean ketemu_2002 = false;

    while (curr_2002 != null) {
        if
        (curr_2002.judul_2002.equalsIgnoreCase(judul_2002)) {
            System.out.println("Lagu ditemukan!");
            System.out.println("Judul : " +
                curr_2002.judul_2002);
            System.out.println("Penyanyi : " +
                curr_2002.penyanyi_2002);
            ketemu_2002 = true;
            break;
        }

        curr_2002 = curr_2002.next_2002;
    }

    if (!ketemu_2002) {
        System.out.println("Lagu tidak ditemukan!");
    }
}

```

1.3 Kelas MainProgram_2511532002

```
package tugasPekan6_2511532002;

import java.util.Scanner;

public class MainProgram_2511532002 {

    public static void main(String[] args) {
        Scanner input_2002 = new Scanner(System.in);
        Musik_2511532002 playlist_2002 = new Musik_2511532002();

        int pilihan_2002;

        System.out.println("\n=== Playlist Musik NIM: 2511532002  

        ===");

        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");

        do {
            System.out.print("\nPilihan : ");

            pilihan_2002 = input_2002.nextInt();
            input_2002.nextLine();

            switch (pilihan_2002) {

                case 1:
                    System.out.print("Judul Lagu : ");
                    String judul_2002 = input_2002.nextLine();

                    System.out.print("Penyanyi : ");
                    String penyanyi_2002 = input_2002.nextLine();

                    playlist_2002.tambahLagu_2002(judul_2002,
                    penyanyi_2002);

                    break;

                case 2:
                    playlist_2002.hapusLaguAwal_2002();
                    break;

                case 3:
                    playlist_2002.tampilMaju_2002();
                    break;

                case 4:
                    playlist_2002.tampilMundur_2002();
                    break;

                case 5:
                    System.out.print("Masukkan judul lagu yang dicari  

                    : ");

                    String cari_2002 = input_2002.nextLine();

                    playlist_2002.cariLagu_2002(cari_2002);
                    break;

                case 6:
                    System.out.println("Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
```

```

    } while (pilihan_2002 != 6);

    input_2002.close();
}
}

```

Penjelasan:

- 1) *package tugasPekan6_2511532002;*
Menjelaskan posisi file berada pada package bernama “*tugaspekan6_2511532002*”.
- 2) *public class MainProgram_2511532002 {*
Merupakan *class* utama (*driver*) yang digunakan untuk menjalankan program.
- 3) *System.out.println("1. Tambah Lagu");*
System.out.println("2. Hapus Lagu Pertama");
System.out.println("3. Lihat Playlist (Maju)");
System.out.println("4. Lihat Playlist (Mundur)");
System.out.println("5. Cari Lagu");
System.out.println("6. Keluar");

Syntax tersebut digunakan untuk menampilkan daftar menu program playlist musik. Menu berisi pilihan operasi yang dapat dijalankan pengguna pada *Double Linked List*.

- 4) *case 1:*
System.out.print("Judul Lagu : ");
String judul_2002 = input_2002.nextLine();
System.out.print("Penyanyi : ");
String penyanyi_2002 = input_2002.nextLine();
playlist_2002.tambahLagu_2002(judul_2002,
penyanyi_2002);
break;

case 2:

playlist_2002.hapusLaguAwal_2002());

break;

case 3:

playlist_2002.tampilMaju_2002());

break;

case 4:

playlist_2002.tampilMundur_2002());

break;

case 5:

*System.out.print("Masukkan judul lagu yang dicari
: ");*

String cari_2002 = input_2002.nextLine();

playlist_2002.cariLagu_2002(cari_2002);

break;

case 6:

System.out.println("Program selesai.");

break;

default:

System.out.println("Pilihan tidak valid!");

}

} while (pilihan_2002 != 6);

Percabangan *switch-case* digunakan untuk menjalankan operasi sesuai pilihan menu yang dimasukkan pengguna.

- i. *case 1* digunakan untuk menambahkan lagu baru ke playlist dengan meminta input judul lagu dan penyanyi, kemudian data dikirim ke method *tambahLagu_2002()*.
- ii. *case 2* digunakan untuk menghapus lagu pertama pada playlist menggunakan method *hapusLaguAwal_2002()*.
- iii. *case 3* digunakan untuk menampilkan playlist secara maju dari awal ke akhir menggunakan method *tampilMaju_2002()*.
- iv. *case 4* digunakan untuk menampilkan playlist secara mundur dari akhir ke awal menggunakan *method*
tampilMundur_2002().
- v. *case 5* digunakan untuk mencari lagu berdasarkan judul dengan memanggil method *cariLagu_2002()*.
- vi. *case 6* digunakan untuk mengakhiri program.
- vii. *default* digunakan untuk menampilkan pesan kesalahan apabila pengguna memasukkan pilihan menu yang tidak tersedia.

2. Hasil

```
Console X
<terminated> MainProgram_2511532002

=== Playlist Musik NIM: 2511532002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan : 1
Judul Lagu : Japanese Denim
Penyanyi : Daniel Caesar
Lagu berhasil ditambahkan!

Pilihan : 1
Judul Lagu : Thank You
Penyanyi : Dido
Lagu berhasil ditambahkan!

Pilihan : 1
Judul Lagu : double take
Penyanyi : Dhruv
Lagu berhasil ditambahkan!

Pilihan : 3

=== Playlist Maju ===
Judul : Japanese Denim
Penyanyi : Daniel Caesar

Judul : Thank You
Penyanyi : Dido

Judul : double take
Penyanyi : Dhruv

Pilihan : 4

=== Playlist Mundur ===
Judul : double take
Penyanyi : Dhruv

Judul : Thank You
Penyanyi : Dido

Judul : Japanese Denim
Penyanyi : Daniel Caesar

Pilihan : 2
Lagu "Japanese Denim" berhasil dihapus.

Pilihan : 5
Masukkan judul lagu yang dicari : Thank You
Lagu ditemukan!
Judul : Thank You
Penyanyi : Dido

Pilihan : 6
Program selesai.
```

3. Kesimpulan

Berdasarkan tugas program yang telah dibuat, dapat disimpulkan bahwa struktur data *Double Linked List* berhasil diimplementasikan menggunakan bahasa pemrograman *Java* pada program playlist musik. Setiap data lagu disimpan dalam bentuk *node* yang memiliki *pointer next* dan *prev*, sehingga data dapat ditelusuri dari dua arah, yaitu maju dan mundur.

Program yang dibuat mampu menjalankan berbagai operasi dasar *Double Linked List*, seperti menambahkan lagu, menghapus lagu pertama, menampilkan *playlist* secara maju dan mundur, serta mencari lagu berdasarkan judul. Dengan pembuatan program ini, penulis dapat memahami cara kerja *Double Linked List* dan penerapannya dalam pengelolaan data secara dinamis.